



Vetores (Arrays)

Vários valores numa variável só · percorrendo com for · procedimentos

7	9	6	8	10
---	---	---	---	----

[0] [1] [2] [3] [4]

```
$ #define TAM 5  int notas[TAM];
```

>_

O que é um vetor?

Um vetor é uma variável que guarda VÁRIOS valores do mesmo tipo, em posições numeradas (índices).

Sem vetor (5 variáveis soltas):

```
int n1, n2, n3, n4, n5;
```

Com vetor (uma variável, 5 posições):

```
int notas[5];
```

- Todos os valores são do mesmo tipo.
- O tamanho é fixo (definido na criação).
- Cada valor é acessado pelo seu índice.

Pense numa fileira de gavetas numeradas:

notas

7	9	6	8	10
[0]	[1]	[2]	[3]	[4]

cada gaveta guarda uma nota; o número embaixo é o índice.

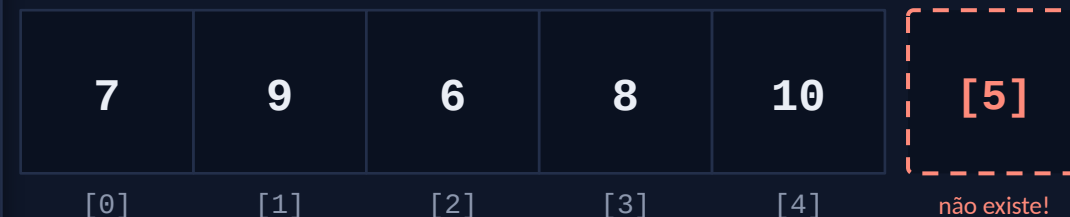


Índices começam em 0 — e o #define TAM 5

O primeiro índice é 0 e o último é TAM - 1. Usamos #define para dar nome ao tamanho.

vetor.c

```
#define TAM 5      // sem ; e sem =  
  
int notas[TAM];    // cria 5 posições: [0] a [4]  
notas[0] = 7;       // primeira posição  
notas[4] = 10;      // última posição
```



Por que #define?

dá um nome ao tamanho. Se um dia o vetor mudar de 5 para 10, você troca em UM lugar só — e todo o código acompanha.

Cuidado clássico: com TAM = 5, acessar notas[5] estoura o vetor (não existe a 6ª gaveta).



Onde usamos vetores?

Sempre que precisamos guardar uma LISTA de valores do mesmo tipo:

Notas de alunos

as 5 notas de uma prova

Temperaturas

a temperatura de cada dia da semana

Carrinho de compras

os itens que o usuário escolheu

Placar do jogo

os pontos de cada rodada

Lista de nomes

os nomes de uma turma

Pixels de uma imagem

as cores, ponto a ponto

A regra: mesma “coisa” repetida muitas vezes → vetor + for para percorrer.

>_

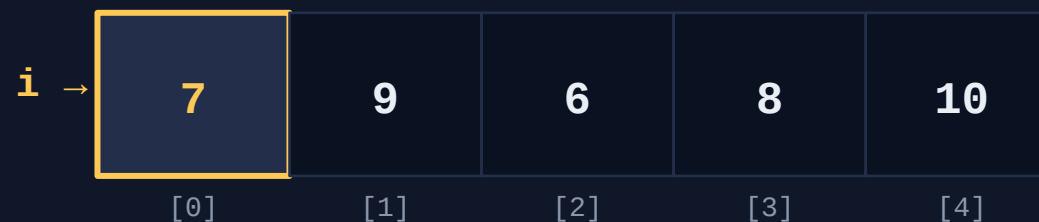
Percorrendo o vetor com for

O índice `i` do `for` (Aula 03) é usado para visitar cada posição: de 0 até `TAM - 1`.

● ● ● `percorrer.c`

```
for (int i = 0; i < TAM; i++) {  
    printf("%d ", notas[i]);  
}
```

`i` começa em 0 e vai até 4 (`TAM - 1`). Em cada volta, `notas[i]` é a posição atual.



`i = 0` `notas[0] = 7`

`i = 1` `notas[1] = 9`

`i = 4` `notas[4] = 10`

`i = 5` `para (5 < 5 é falso)`



Procedimentos: carregarVetor e exibirVetor

Dois procedimentos (void) que recebem o vetor e o tamanho, e percorrem com for.

● ● ● carregar.c

```
void carregarVetor(int v[], int tam) {  
    for (int i = 0; i < tam; i++) {  
        printf("Valor %d: ", i);  
        scanf("%d", &v[i]);  
    }  
}
```

● ● ● exibir.c

```
void exibirVetor(int v[], int tam) {  
    for (int i = 0; i < tam; i++)  
        printf("%d ", v[i]);  
    printf("\n");  
}
```

Detalhe importante (lembra da Aula 04?)

O vetor é passado por REFERÊNCIA automaticamente — o nome `v` já é o endereço. Por isso o `carregarVetor` consegue preencher o vetor de verdade. E o `&` do `scanf` continua sendo o endereço de cada posição.

>_

Exemplo: inverterVetor (com teste de mesa)

Invertemos trocando as pontas: $[0] \leftrightarrow [4]$, depois $[1] \leftrightarrow [3]$. O do meio fica parado.

●●● inverter.c

```
void inverterVetor(int v[], int tam) {  
    for (int i = 0; i < tam / 2; i++) {  
        int temp = v[i];  
        v[i] = v[tam - 1 - i];  
        v[tam - 1 - i] = temp;  
    }  
}
```

É a mesma troca (swap) da Aula 04 — agora entre posições do vetor.

Teste de mesa (TAM = 5)

início:

1	2	3	4	5
---	---	---	---	---

i=0:

5	2	3	4	1
---	---	---	---	---

troca $[0] \leftrightarrow [4]$

i=1:

5	4	3	2	1
---	---	---	---	---

troca $[1] \leftrightarrow [3]$

resultado: 5 4 3 2 1

i vai só até $\text{tam}/2$ ($=2$): visita 0 e 1. O meio $[2]$ não precisa trocar.

>_

Juntando tudo no main

Com `#define TAM 5`, o main só chama os procedimentos. Fica limpo e fácil de ler.

main.c

```
#include <stdio.h>
#define TAM 5
// (carregarVetor, exibirVetor e inverterVetor definidos acima)

int main() {
    int notas[TAM];
    carregarVetor(notas, TAM);
    printf("Original: ");
    exibirVetor(notas, TAM);
    inverterVetor(notas, TAM);
    printf("Invertido: ");
    exibirVetor(notas, TAM);
    return 0;
}
```

Saída (ex.: 1 2 3 4 5)

Original: 1 2 3 4 5
Invertido: 5 4 3 2 1

Repare

o main lê como uma receita: carregar → exibir → inverter → exibir. Cada procedimento cuida de um pedaço, e todos percorrem o vetor com for.



Exercícios para fazer em sala

Em todos: use `#define TAM 5` e percorra o vetor com `for`. Faça o teste de mesa antes de rodar.

1

Somar

Some todos os elementos do vetor e mostre o total.

2

Média

Calcule a média dos 5 elementos (soma / TAM).

3

dobrarVetor

Procedimento que multiplica cada elemento por 2 (altera o vetor).

4

Contar pares

Conte quantos elementos são pares ($v[i] \% 2 == 0$).

5

Maior valor

Encontre e mostre o maior valor do vetor.

>_

Gabarito (trechos principais)

ex1.c

```
// 1) Somar
int soma = 0;
for (int i=0; i<TAM; i++)
    soma += v[i];
```

ex2.c

```
// 2) Média
float media = (float) soma / TAM;
printf("%.2f", media);
```

ex3.c

```
// 3) dobrarVetor (procedimento)
void dobrarVetor(int v[], int tam){
    for (int i=0; i<tam; i++)
        v[i] = v[i] * 2;
}
```

ex4.c

```
// 4) Contar pares
int pares = 0;
for (int i=0; i<TAM; i++)
    if (v[i] % 2 == 0)
        pares++;
```

ex5.c

```
// 5) Maior valor
int maior = v[0];
for (int i=1; i<TAM; i++)
    if (v[i] > maior)
        maior = v[i];
```

Padrão comum: um acumulador (soma, pares, maior) fora do for, atualizado a cada volta.

>_

Recapitulando

- Vetor: uma variável com várias posições do mesmo tipo.
- Índices vão de 0 até TAM - 1 (nunca acesse TAM).
- #define TAM 5 dá nome ao tamanho — muda em um lugar só.
- Percorremos o vetor com for (i de 0 a TAM - 1).
- O vetor vai por referência: procedimentos alteram o original.
- carregarVetor, exibirVetor e inverterVetor deixam o main limpo.

O padrão que se repete

```
for (int i=0; i<TAM; i++)  
    ...usar v[i]
```

Decore esse esqueleto: quase todo exercício de vetor começa assim.

Um vetor + um for: é assim que guardamos e processamos listas de dados.